

DailyDeck

Architecture & Backend Structure Guide

System Blueprint • File Responsibilities • Integration Flows

1. Executive Architecture Overview

DailyDeck operates on a decoupled client-server architecture designed specifically for educational slide automation. The system consists of two primary runtime engines: a high-performance frontend client with local state management, and a cloud-native Google Apps Script backend API that controls Google Drive and Google Slides.

Core Architecture Components:

- **Frontend Client (SPA):** Modular ES6 application running in the teacher's browser. Coordinates section management, state persistence, 16:9 real-time slide preview, and recipe construction.
- **Web Server Layer (server.js):** Lightweight Node.js Express server providing zero-cache asset delivery and single-page routing over HTTP/3000.
- **Slide Assembly Engine (Code.gs):** Google Apps Script cloud microservice (doPost API). Executes inside Google Workspace to clone master templates, substitute dynamic text tokens, inject speaker notes, convert to PPTX, and manage permissions.
- **Data Compilation Pipeline (recipe.js):** The abstraction bridge transforming high-level UI deck models into atomic, idempotent slide-assembly command objects.

2. End-to-End System Data Flow

DATA INTERACTION & ASSEMBLY FLOW

1. Configure

Teacher defines items in UI

2. Compile

recipe.js builds JSON payload

3. Transmit

POST request to Apps Script
URL

4. Assemble

Code.gs clones & mutates
Slides

!• **Response Payload:** `{ status: "success", url: "https://docs.google.com/...", copyUrl: "...", pptxUrl: "..." }`

3. Primary Backend & Server Files

server.js

NODE.JS / EXPRESS SERVER

Purpose: Application host and static file distributor.

Express 4 server running on host 0.0.0.0, port 3000. Implements strict no-cache headers (no-store, must-revalidate) during development to prevent stale script caching. Handles SPA wildcard routing fallback to index.html.

Code.gs

GOOGLE APPS SCRIPT BACKEND ENGINE

Purpose: The remote microservice orchestrating Google Drive and Google Slides operations.

Implements doPost(e) HTTP endpoint. Parses JSON assembly recipes, invokes DriveApp to duplicate the master presentation template into the target folder, indexes slide speaker notes for [tags], duplicates matching slides, executes text substitution, and injects speaker notes.

4. Comprehensive File Directory & Roles

■ `src/export/recipe.js`

COMPILATION ENGINE

Purpose: Transforms state items into the standardized slide assembly schema.

Translates timeline items across active days into atomic slide commands ({ notelId, replacements, injectNotes }). Computes sound pyramid breakdowns, pairs, sentence type slides, and metadata speaker notes.

■ `src/state/state.js`

STATE MANAGER & STORE

Purpose: Centralized data repository and localStorage persistence layer.

Maintains state for dual workspaces (Literacy and Numeracy), globalSettings (year level, term, week, templateId, folderId), timelineItems array, and shared custom templates. Serializes data to versioned LocalStorage keys.

■ `src/data/constants.js`

CONSTANTS & CONFIGURATION

Purpose: Immutable system definitions, template identifiers, and grammar dictionaries.

Defines DEFAULT_TEMPLATE_ID, NUMERACY_DEFAULT_TEMPLATE_ID, DAYS list ("Monday"- "Friday"), Sounds-Write phonic units, and definitions for sentence types (Declarative, Imperative, Interrogative, Exclamatory).

■ `src/data/sections.js`

ACTIVITY CATALOG

Purpose: Single source of truth for all activity types and form field schemas.

Defines SECTION_DEFS: activity metadata, icon, background styling, activity type (static, daily, weekly, mixed), and dynamic form field structures used by inspector.js to auto-generate editors.

■ `src/data/starters.js`

CURATED STARTERS

Purpose: Pre-configured turnkey deck starter blueprints.

Defines STARTER_DEFS for Kindergarten Core, Stage 1 Foundation, Stage 2 Phonics & Grammar, and custom decks with pre-populated activity sequences and sample teacher data.

■ `src/data/phonics.js`

CURRICULUM WORD BANK

Purpose: Sounds-Write phonics pool index and level matrix.

Contains target sound mappings, high-frequency words, and level metadata for systematic synthetic phonics progression.

5. UI Controller Files (Interface & Preview)

■ src/main.js

APPLICATION ORCHESTRATOR

Purpose: Main entry point, event listeners, and API network dispatcher.

Initializes workspace toggles (Literacy/Numeracy), accordions, drag-and-drop sequencing via SortableJS, and executes the fetch() POST call to sendRecipeToAppScript(). Controls toast alerts and assembly dialogs.

■ src/ui/inspector.js

FORM BUILDER & TEMPLATE INSPECTOR

Purpose: Dynamic property inspector and template management interface.

Builds form fields reactively based on section schema definitions, synchronizes template dropdown options, manages custom user templates, and triggers state saves.

■ src/ui/timeline.js

DECK SEQUENCER MANAGER

Purpose: Interactive visual deck timeline manager.

Renders ordered activity cards, drag handles, active day badges (e.g., M-W-F), delete triggers, and contextual empty-state guidance.

■ src/ui/preview.js

16:9 SLIDE PREVIEW RENDERER

Purpose: High-fidelity browser-side SVG/HTML slide preview simulation.

Generates instantaneous visual representations of slides as they will appear on Google Slides canvas, supporting day tabs, casing toggles, and sub-slide pagination.

6. Backend JSON Assembly Contract

When the user triggers deck assembly, src/export/recipe.js compiles an explicit JSON recipe payload transmitted directly to Code.gs. The contract follows this schema:

```
{
  "templateId": "12f9hl__t2nghjPeZS01Ag1P-dLmJXXDKzCSVeJT2QbY",
  "deckName": "Kindergarten Term 1 Week 5 Literacy Daily Review",
  "folderId": "1AbC...optional_drive_folder_id",
  "exportPptx": true,
  "slides": [
    {
      "noteId": "[mainTitle]",
      "replacements": { "{yearLevel}": "Kindergarten", "{termNumber}": "1", "{weekNumber}": "5" },
      "injectNotes": "Deck Name: K Review\nAuthor: Teacher\nCreated on: Wed 24 Sep"
    },
    {
      "noteId": "[pyramid3]",
      "replacements": { "{letter1}": "s", "{letter2}": "a", "{letter3}": "t", "{word}": "sat" }
    }
  ]
}
```

7. Backend Security & Google API Integration

- **Zero Backend Credentials Stored:** The Node.js server does not store or process user credentials. All slide creation happens in the user's or organization's Google Workspace domain.
- **Google Drive & Slides Scopes:** Code.gs runs with standard Workspace scopes (DriveApp, SlidesApp, UrlFetchApp) with execute-as-user authorization, ensuring teachers retain full document ownership.
- **Idempotent Assembly:** Each assembly run produces a fresh duplicate of the template presentation without mutating or overwriting the original master library.

